

Terraform Modules - Real-Time Multi-Environment Structure

A practical guide to Terraform modules, why we use them, and a real-world Dev/Stage/Prod structure with code.

1. What is a Terraform module?

A **Terraform module** is a collection of Terraform configuration files grouped together so the **infrastructure code can be reused**. Instead of copying the same resource definitions for Dev, Stage, and Prod, we create the infrastructure logic once and call it from each environment.

```
modules/  
  ec2/  
    main.tf  
    variables.tf  
    outputs.tf
```

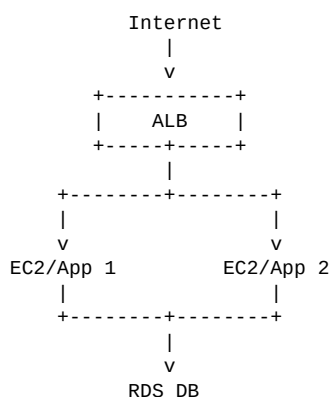
```
Dev   -> EC2 module -> 1 instance  
Stage -> EC2 module -> 2 instances  
Prod  -> EC2 module -> 4 instances
```

2. Why do we use modules?

Benefit	Explanation
Reusability	Write infrastructure logic once and reuse it across environments.
Avoid duplication	Do not copy/paste the same VPC, EC2, ALB, RDS, or IAM code.
Consistency	All environments follow the same infrastructure pattern.
Maintainability	A common change can be made in one module instead of many files.
Standardization	Teams can create approved patterns for networking, compute, databases, etc.
Separation of concerns	Reusable infrastructure logic stays separate from environment-specific configuration.

3. Real-time example

Imagine a payment application deployed on AWS. A typical architecture contains an ALB, application EC2 instances, and an RDS database. The architecture is similar in every environment, but capacity and settings differ.



Setting	Dev	Stage	Prod
EC2 count	1	2	4
EC2 type	t3.micro	t3.small	t3.medium
Database	Small	Medium	Large
Multi-AZ	No	Yes	Yes

Setting	Dev	Stage	Prod
Environment	dev	stage	prod

4. Real-time Terraform project structure

```

terraform-infrastructure/
|
+-- modules/
|   +-- vpc/
|   |   +-- main.tf
|   |   +-- variables.tf
|   |   +-- outputs.tf
|   |
|   +-- security-group/
|   |   +-- main.tf
|   |   +-- variables.tf
|   |   +-- outputs.tf
|   |
|   +-- alb/
|   |   +-- main.tf
|   |   +-- variables.tf
|   |   +-- outputs.tf
|   |
|   +-- ec2/
|   |   +-- main.tf
|   |   +-- variables.tf
|   |   +-- outputs.tf
|   |
|   +-- rds/
|   |   +-- main.tf
|   |   +-- variables.tf
|   |   +-- outputs.tf
|
+-- environments/
|   +-- dev/
|   |   +-- backend.tf
|   |   +-- provider.tf
|   |   +-- main.tf
|   |   +-- variables.tf
|   |   +-- terraform.tfvars
|   |   +-- outputs.tf
|   |
|   +-- stage/
|   |   +-- backend.tf
|   |   +-- provider.tf
|   |   +-- main.tf
|   |   +-- variables.tf
|   |   +-- terraform.tfvars
|   |   +-- outputs.tf
|   |
|   +-- prod/
|   |   +-- backend.tf
|   |   +-- provider.tf
|   |   +-- main.tf
|   |   +-- variables.tf
|   |   +-- terraform.tfvars
|   |   +-- outputs.tf

```

5. Build an EC2 module

modules/ec2/main.tf

```
resource "aws_instance" "this" {
  count = var.instance_count

  ami          = var.ami_id
  instance_type = var.instance_type
  subnet_id    = var.subnet_id

  tags = {
    Name          = "${var.environment}-app-${count.index + 1}"
    Environment = var.environment
  }
}
```

modules/ec2/variables.tf

```
variable "instance_count" {
  description = "Number of EC2 instances"
  type        = number
}

variable "ami_id" {
  description = "AMI ID"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}

variable "subnet_id" {
  description = "Subnet ID"
  type        = string
}

variable "environment" {
  description = "Environment name"
  type        = string
}
```

modules/ec2/outputs.tf

```
output "instance_ids" {
  value = aws_instance.this[*].id
}

output "private_ips" {
  value = aws_instance.this[*].private_ip
}
```

6. Dev environment

environments/dev/main.tf

```
provider "aws" {
  region = "ap-south-1"
}

module "app" {
  source = "../../modules/ec2"

  instance_count = var.instance_count
  ami_id         = var.ami_id
  instance_type  = var.instance_type
  subnet_id     = var.subnet_id
  environment    = var.environment
}
```

environments/dev/variables.tf

```
variable "instance_count" { type = number }
variable "ami_id"         { type = string }
variable "instance_type"  { type = string }
```

```
variable "subnet_id"      { type = string }
variable "environment"    { type = string }
```

environments/dev/terraform.tfvars

```
instance_count = 1
ami_id         = "ami-0123456789abcdef"
instance_type  = "t3.micro"
subnet_id      = "subnet-12345678"
environment    = "dev"
```

7. Stage environment

environments/stage/main.tf

```
provider "aws" {
  region = "ap-south-1"
}

module "app" {
  source = "../../modules/ec2"

  instance_count = var.instance_count
  ami_id         = var.ami_id
  instance_type  = var.instance_type
  subnet_id      = var.subnet_id
  environment    = var.environment
}
```

environments/stage/terraform.tfvars

```
instance_count = 2
ami_id         = "ami-0123456789abcdef"
instance_type  = "t3.small"
subnet_id      = "subnet-87654321"
environment    = "stage"
```

8. Production environment

environments/prod/main.tf

```
provider "aws" {
  region = "ap-south-1"
}

module "app" {
  source = "../../modules/ec2"

  instance_count = var.instance_count
  ami_id         = var.ami_id
  instance_type  = var.instance_type
  subnet_id      = var.subnet_id
  environment    = var.environment
}
```

environments/prod/terraform.tfvars

```
instance_count = 4
ami_id         = "ami-0123456789abcdef"
instance_type  = "t3.medium"
subnet_id      = "subnet-99887766"
environment    = "prod"
```

9. Module-to-module communication

A larger environment can call several modules. Outputs from one module can be passed into another module.

```
module "vpc" {
  source = "../../modules/vpc"

  environment = var.environment
  cidr_block  = var.vpc_cidr
}

module "security_group" {
  source = "../../modules/security-group"

  environment = var.environment
  vpc_id      = module.vpc.vpc_id
}

module "app" {
  source = "../../modules/ec2"

  environment      = var.environment
  instance_count  = var.instance_count
  instance_type   = var.instance_type
  ami_id          = var.ami_id

  subnet_id = module.vpc.private_subnet_id
}

module "database" {
  source = "../../modules/rds"

  environment = var.environment
  vpc_id      = module.vpc.vpc_id
}
```

The VPC module can expose an output:

```
output "private_subnet_id" {
  value = aws_subnet.private.id
}
```

The EC2 module consumes it using `module.vpc.private_subnet_id`. This creates a clean dependency between modules.

10. What changes between environments?

```
MODULE CODE
|
| Same reusable infrastructure logic
|
+---- DEV    -> 1 x t3.micro
|
+---- STAGE  -> 2 x t3.small
|
+---- PROD   -> 4 x t3.medium
```

The module stays reusable.
The environment supplies different inputs.

11. Why not copy the code?

Without modules, you may have separate EC2/VPC/RDS definitions in Dev, Stage, and Prod. A security or configuration change then has to be repeated in multiple places. With a shared module, common infrastructure logic is maintained centrally and environments consume it.

12. Module vs Environment

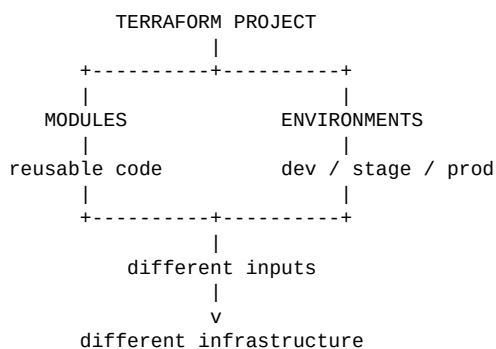
Module	Environment
Reusable infrastructure code	Specific deployment configuration
Contains resources	Calls modules

Module	Environment
Generic and parameterized	Environment-specific
Example: modules/ec2	Example: environments/prod
Defines how infrastructure is created	Defines which configuration to use

13. Interview-ready answer

Terraform modules are reusable collections of Terraform configuration that encapsulate infrastructure resources. We use modules to avoid code duplication, standardize infrastructure, improve maintainability, and reuse the same infrastructure pattern across multiple environments. For example, we can create an EC2 module containing the EC2 resource, variables, and outputs. Dev, Stage, and Prod can all call the same module while passing different values such as instance count, instance type, AMI, and subnet. This keeps infrastructure logic reusable while keeping environment-specific configuration separate.

14. Key mental model



15. Important real-time point: Terraform state

Keep Terraform state separated between environments. For example, Dev and Prod should not accidentally share the same state. Teams commonly use separate backend state keys/paths or another deliberate state-isolation strategy. The exact backend design depends on the organization.